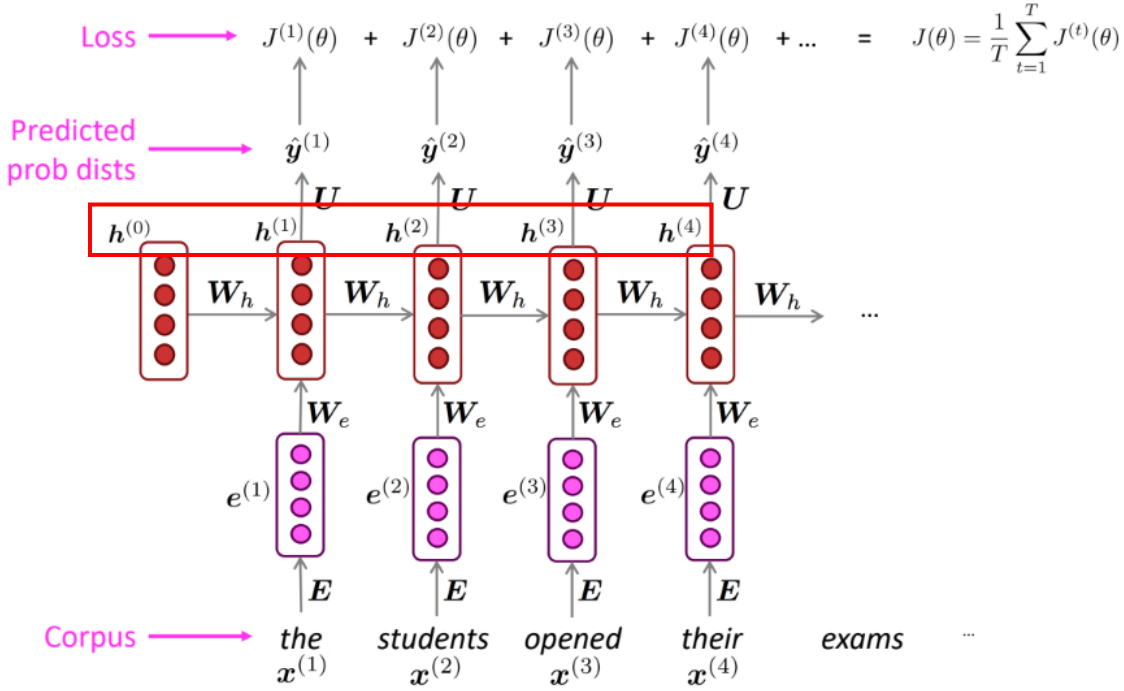
# CS224n & Pytorch Practice

2026.04.23

Hayeong Jung

# Review

- **Language Modeling**
  - A System that predicts the Next word
  - Subcomponent of NLP Tasks
    - Generating
    - Estimating $\boldsymbol{P}$ of test
- **RNN**
  - Structure
  - Pros & Cons
  - Training
    - Loss
    - Backpropagation



# Lecture 6: Sequence to Sequence Models

- **Evaluation Metric**
  - Perplexity $\text{perplexity} = \prod_{t=1}^{T}\left(\frac{1}{P_{LM}(x^{(t+1)}|x^{(t)},\dots,x^{(1)})}\right)^{1/T} = \prod_{t=1}^{T}\left(\frac{1}{\hat{y}^{(t)}_{x_{t+1}}}\right)^{1/T}$
  - Cross-Entropy
- **Problems of RNN**
  - Exploding Gradients
    - Eigenvalues of $W_h > 1$
    - Too Big Step
  - Vanishing Gradients
    - Eigenvalues of $W_h < 1$
    - Can't Learn Long-distance Dependency

# Lecture 6

- **Solution of Gradients Problem**
  - Gradient Clipping
    - For Gradient Exploding Problem
    - Scale Down when $\boldsymbol{Gradient} \geq \boldsymbol{threshold}$

**Algorithm 1** Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
**if** $\|\hat{\mathbf{g}}\| > threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \boxed{\frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}}$
**end if**

Crude but Work Well!

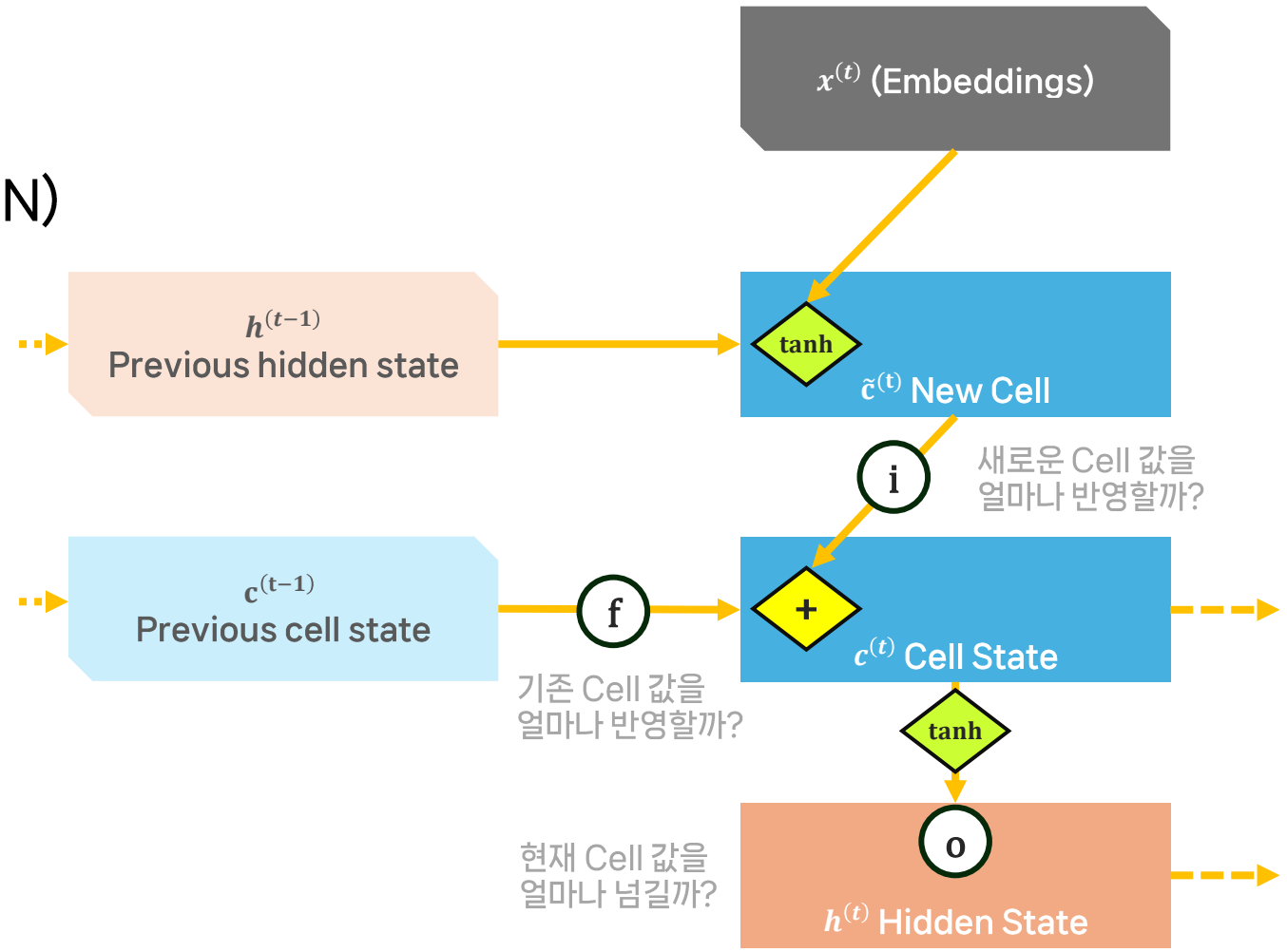
    - Same Direction, Smaller Step
  - Add Memory
    - For Gradient Vanishing Problem
    - Hard to Preserve Past Information

$$\boldsymbol{h}^{(t)} = \boldsymbol{\sigma}\left(\underbrace{\boldsymbol{W}_h \boldsymbol{h}^{(t-1)}}_{\text{Previous Hidden State}} + \underbrace{\boldsymbol{W}_x \boldsymbol{x}^{(t)}}_{\text{Current Word Embedding}} + \boldsymbol{b}\right)$$

- **LSTM** (Long Short-Term Memory RNN)
  - Gates
    - Forget Gate $\boldsymbol{f}^{(t)} = \boldsymbol{\sigma}\left(\mathbf{W_f h^{(t-1)}} + \mathbf{U_f x^{(t)}} + \mathbf{b_f}\right)$
    - Input Gate $\boldsymbol{i}^{(t)} = \boldsymbol{\sigma}\left(\mathbf{W_i h^{(t-1)}} + \mathbf{U_i x^{(t)}} + \mathbf{b_i}\right)$
    - Output Gate $\boldsymbol{o}^{(t)} = \boldsymbol{\sigma}\left(\mathbf{W_o h^{(t-1)}} + \mathbf{U_o x^{(t)}} + \mathbf{b_o}\right)$
  - Cells
    - New Cell
      $\tilde{\mathbf{c}}^{(t)} = \tanh\left(\mathbf{W_c h^{(t-1)}} + \mathbf{U_c x^{(t)}} + \mathbf{b_c}\right)$
    - Cell State
      $\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \boldsymbol{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)}$
    - Hidden State
      $\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh \mathbf{c}^{(t)}$

# Lecture 6

- **LSTM** (Long Short-Term Memory RNN)
  - Gates
    - $f^{(t)} = \sigma\left(W_f h^{(t-1)} + U_f x^{(t)} + b_f\right)$
    - $i^{(t)} = \sigma\left(W_i h^{(t-1)} + U_i x^{(t)} + b_i\right)$
    - $o^{(t)} = \sigma\left(W_o h^{(t-1)} + U_o x^{(t)} + b_o\right)$
  - Cells
    - New Cell
      $\tilde{c}^{(t)} = \tanh\left(W_c h^{(t-1)} + U_c x^{(t)} + b_c\right)$
    - Cell State
      $c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$
    - Hidden State
      $h^{(t)} = o^{(t)} \odot \tanh c^{(t)}$

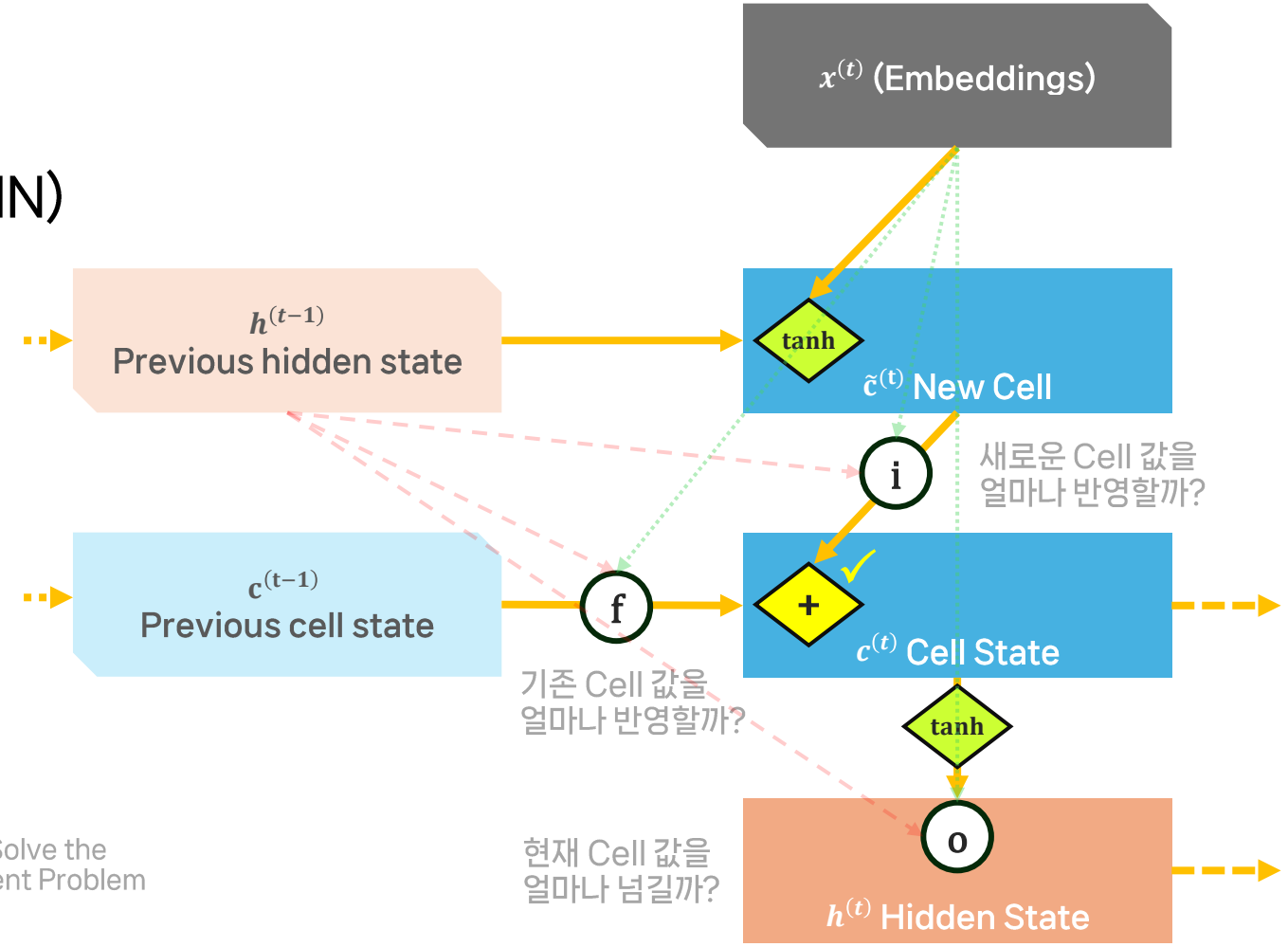


# Lecture 6

- **LSTM** (Long Short-Term Memory RNN)
  - Gates
    - $f^{(t)} = \sigma\left(W_f h^{(t-1)} + U_f x^{(t)} + b_f\right)$
    - $i^{(t)} = \sigma\left(W_i h^{(t-1)} + U_i x^{(t)} + b_i\right)$
    - $o^{(t)} = \sigma\left(W_o h^{(t-1)} + U_o x^{(t)} + b_o\right)$

      Context(or Signal) for the Best Choice
      to Preserve the Adequate Information
  - Cells
    - New Cell
      $\tilde{c}^{(t)} = \tanh\left(W_c h^{(t-1)} + U_c x^{(t)} + b_c\right)$
    - Cell State
      $c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$

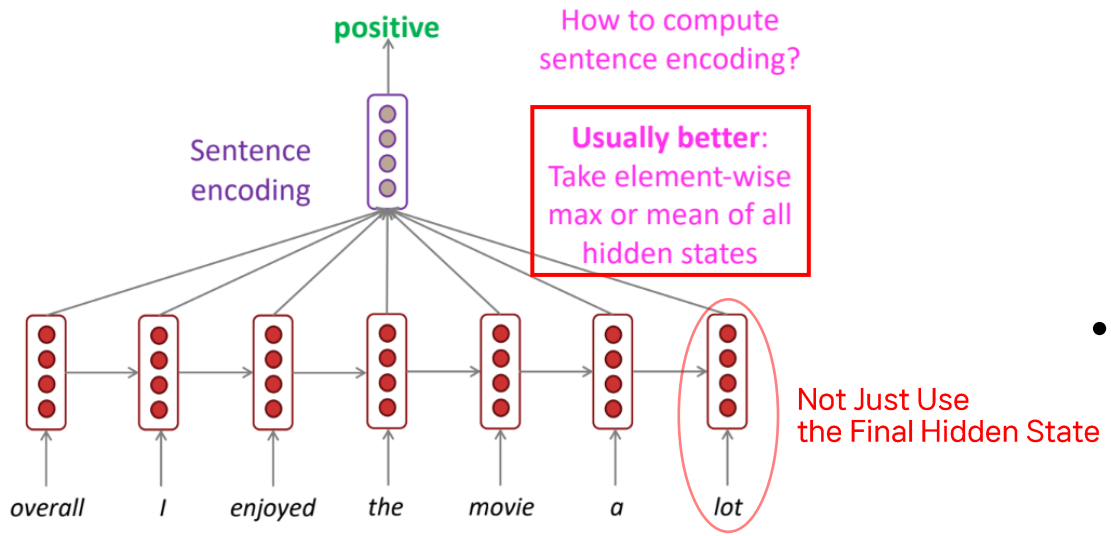
      Key Point to Solve the
      Vanishing Gradient Problem
    - Hidden State
      $h^{(t)} = o^{(t)} \odot \tanh c^{(t)}$



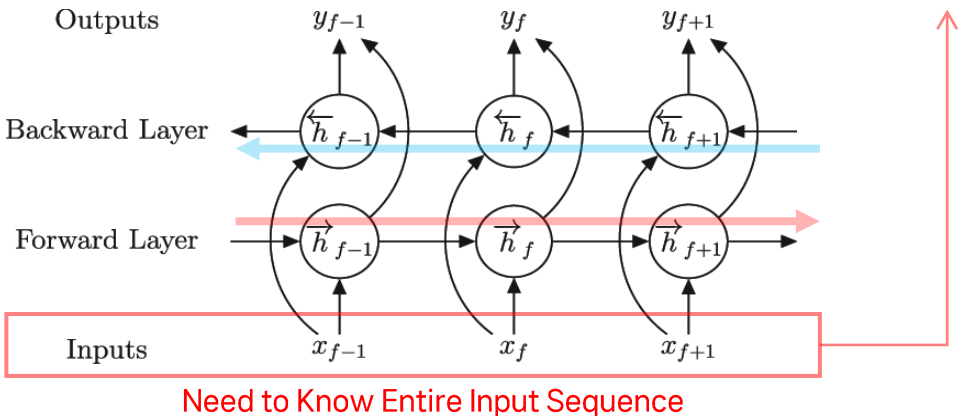

$$\frac{\partial h^{(t)}}{\partial h^{(t-1)}} = \mathrm{diag}\left(\sigma'\left(z^{(t)}\right)\right) \cdot W_h \quad \text{VS} \quad \frac{\partial c^{(t)}}{\partial c^{(t-1)}} \approx f^{(t)}$$

# Lecture 6

- RNN Uses
  - Named Entity Recognition(NER)
  - Sentiment Classification
  - Speech Recognition
  - Machine Translation
  - Summarization



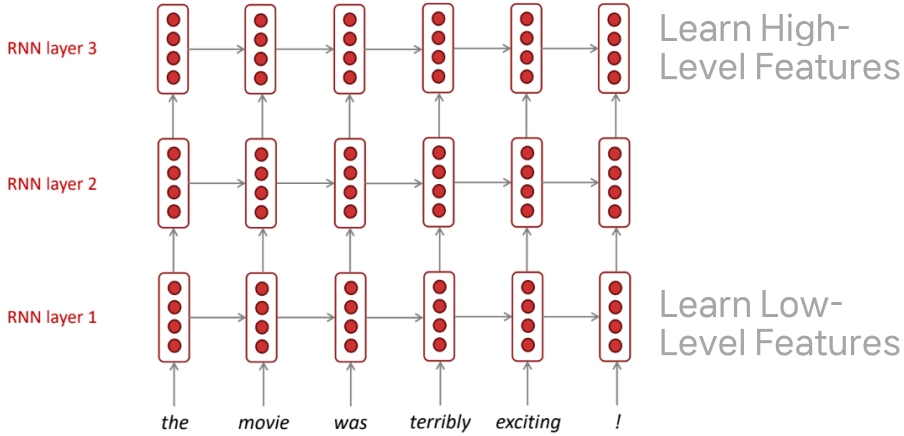

- Bidirectional RNNs
  - Usages
    - Language Analysis O
    - Generating Text, Language Modeling X




- Cf., BERT
  - (Bidirectional Encoder Representation from Transformer)

# Lecture 6

- **Multi-layer RNNs**
  - Structure




  - Advantage
    - Can Compute More Complex Representations
    - Usually Use Multi-layer

- **Machine Translation**
  - Statistical Machine Translation
    - Use Bayes Rule

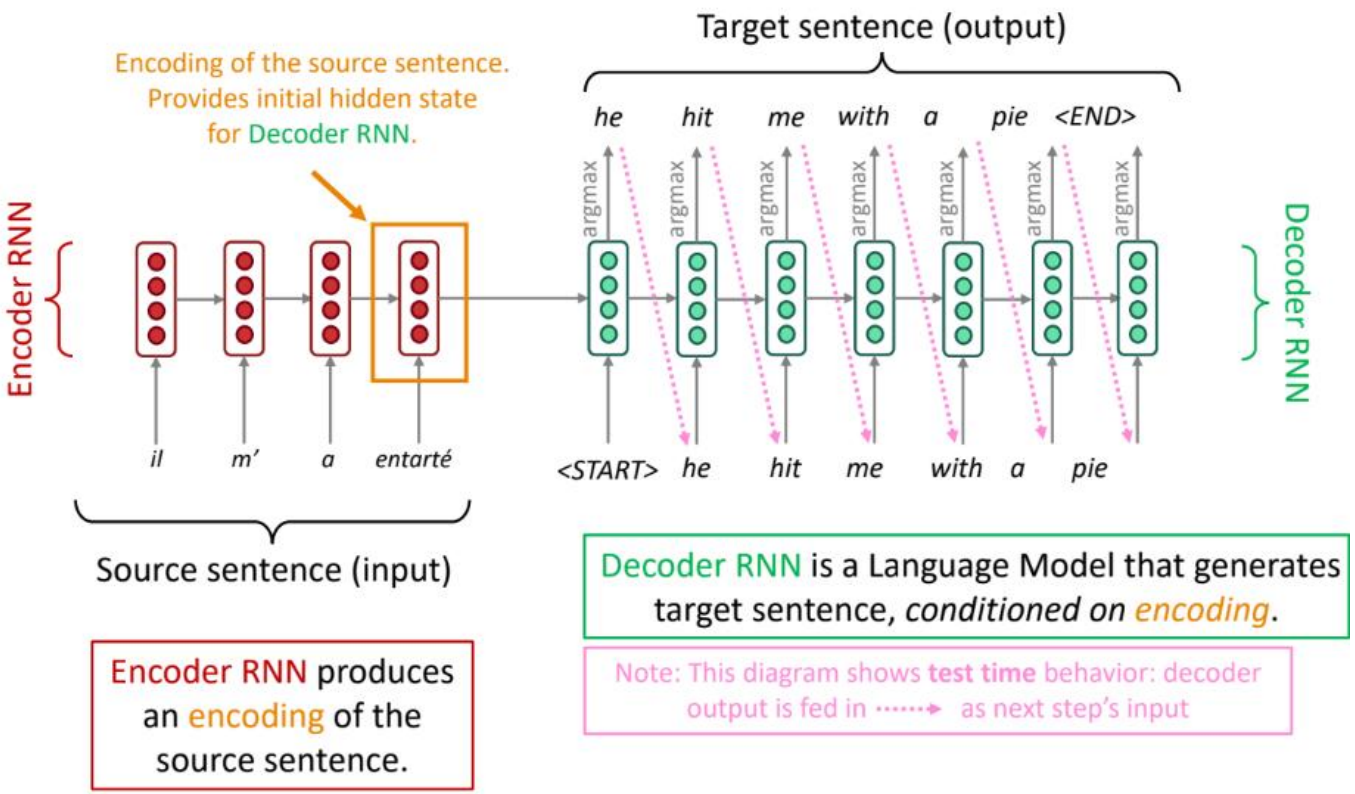
$$P(y \mid x) = \frac{P(x \mid y)P(y)}{P(x)},$$

$$\arg\max_{y} P(x \mid y)\, P(y)$$

  - Neural Machine Translation
    - Single End-to-End Neural Network
    - 2 RNNs
      - Encoder
      - Decoder

# Lecture 6

- Seq2Seq Model
  - Conditional LM
    - Conditioned on Source $X$
  - Optimized as a Single System
    - End-to-End Backprop
  - Usages
    - Summarization
      long text
      → short text
    - Dialogue
      previous utterances
      → next utterance
    - Parsing
      input text
      → output parse as sequence
    - Code generation
      natural language
      → Python code

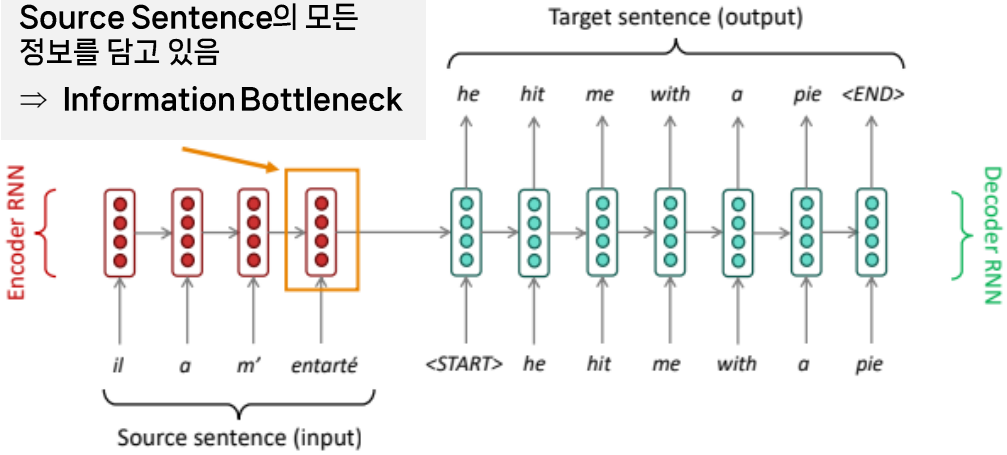


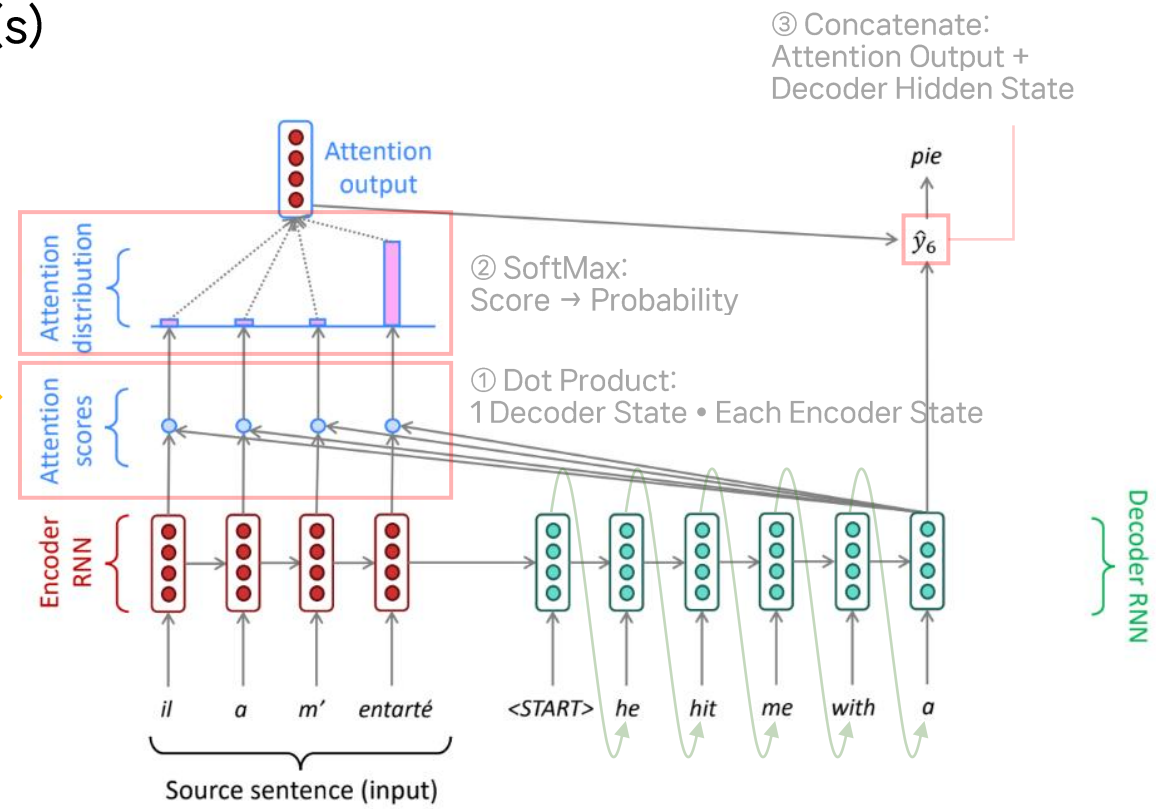
# Lecture 7: Attention

- **Evaluation of Machine Translation**
  - BLEU Measure
    - Machine Translation ↔ Human Translation(s)
    - N-Gram Precision: Useful but Imperfect
- **Attention**
  - Solution of Bottleneck Problem

Encoder의 마지막 State에 Source Sentence의 모든 정보를 담고 있음

⇒ Information Bottleneck

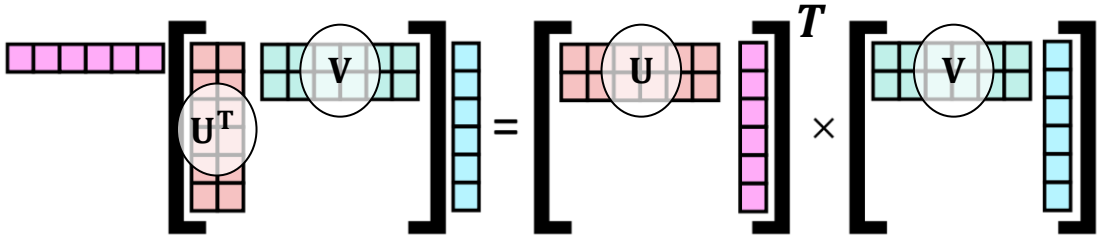


# Lecture 7

- **Attention Equation**
  - Encoder Hidden State
    $\boldsymbol{h_1}, \dots, \boldsymbol{h_N} \in \mathbb{R}^{\mathbb{h}}$
  - Decoder Hidden State
    $\boldsymbol{s_t} \in \mathbb{R}^{\mathbb{h}}$
  - Attention Scores
    $\boldsymbol{e^t} = [\boldsymbol{s_t^T h_1}, \dots, \boldsymbol{s_t^T h_N}] \in \mathbb{R}^{\mathbb{N}}$
  - Attention Distribution
    $\boldsymbol{\alpha^t} = \mathbf{softmax}(\boldsymbol{e^t}) \in \mathbb{R}^{\mathbb{N}}$
  - Attention Output
    $\boldsymbol{a_t} = \sum_{i=1}^{N} \boldsymbol{\alpha_i^t h_i} \in \mathbb{R}^{\mathbb{h}}$
  - Concatenation
    $[\boldsymbol{a_t}; \boldsymbol{s_t}] \in \mathbb{R}^{2\mathbb{h}}$

- **Attention Variants**
  - Several ways to Compute $\boldsymbol{e} \in \mathbb{R}^{\boldsymbol{N}}$
    $(\mathbf{h_1}, \dots, \mathbf{h_N} \in \mathbb{R}^{\mathbb{d}_1}, \mathbf{s} \in \mathbb{R}^{\mathbb{d}_2})$
    - Basic Dot Product Attention
      $\boldsymbol{e^i} = \boldsymbol{S^T h_i}$ $(\mathbf{when}\ \mathbb{d}_1 = \mathbb{d}_2)$
    - Multiplicative Attention
      $e_i = \mathbf{s^T W h_i} \in \mathbb{R}$ $(\mathbf{when}\ \mathbf{W} \in \mathbb{R}^{\mathbb{d}_2 \times \mathbb{d}_1})$
    - Reduced Multiplicative Attention
      $e_i = \mathbf{s^T}(\mathbf{U^T V})\mathbf{h_i} = (\mathbf{Us})^T(\mathbf{Vh_i})$
      $(\mathbf{U} \in \boldsymbol{R}^{\boldsymbol{k} \times \boldsymbol{d_2}}, \mathbf{V} \in \boldsymbol{R}^{\boldsymbol{k} \times \boldsymbol{d_1}}, \boldsymbol{k} \ll \boldsymbol{d_1}, \boldsymbol{d_2})$

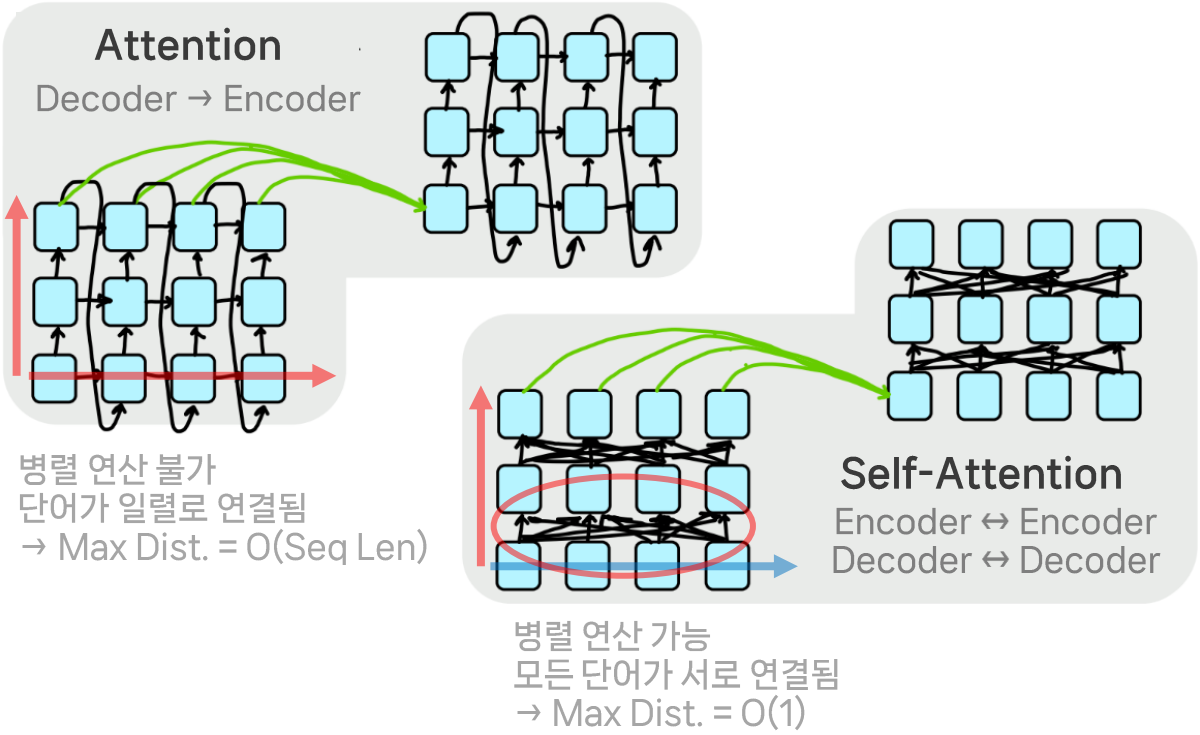


# Lecture 8: Self-Attention and Transformers

- **Before Transformer**
  - Strategy (w/ LSTM)
    - Encoder: Bidirectional LSTM
    - Decoder: Unidirectional LSTM
    - + Attention
  - Limitation
    - Linear Interaction Distance
      - Sequence Length = $N$
      - Take $O(N)$ Steps to Interact
    - Lack of Parallelizability
      - Compute Sequentially

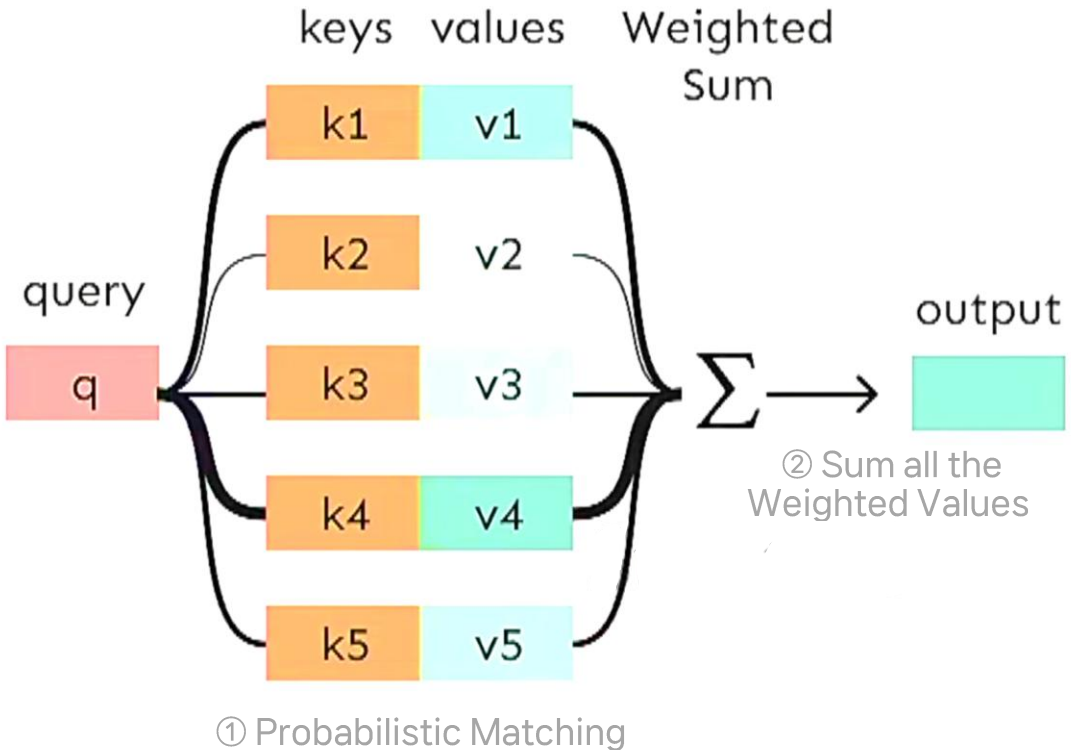
    ⇒ *Dependence on Time (Step)*

- **Attention vs Self-Attention**



# Lecture 8

- **Self-Attention in Encoder**




- Attention Mechanism
  - Calculate queries, keys, and values (w/ Embedding Stack $X$)
    $Q = XW^Q \quad K = XW^K \quad V = XW^V$
  - Calculate attention scores between Q & K
    $$E = QK^T$$
  - Normalize attention scores
    $$A = \mathrm{softmax}(E)$$
  - Take a weighted sum of values
    $$Output = AV$$
  - Final Output
    $$Output = \mathrm{softmax}(QK^T)V$$

# Lecture 8

- Sequence Order Problem

Man eats small Dinosaur
=
Small Dinosaur eats Man

**Order Doesn't Impact!**

  - Sinusoidal Position Representation
  - Learned Absolute Position Representation

$$\boldsymbol{h_i} = \boldsymbol{x_i} + \boldsymbol{p_i} \quad (\boldsymbol{p_i}\text{: position vector})$$

  - Relative Position Encoding

$$\boldsymbol{e_{ij}} = \frac{\boldsymbol{x_i W^Q}\left(\boldsymbol{x_j W^K} + \boldsymbol{a_{ij}^K}\right)^{\boldsymbol{T}}}{\sqrt{\boldsymbol{d_z}}}$$

$$\boldsymbol{z_i} = \sum_{\boldsymbol{j=1}}^{\boldsymbol{n}} \boldsymbol{\alpha_{ij}}\left(\boldsymbol{x_j W^V} + \boldsymbol{a_{ij}^V}\right)$$

- Training Tricks
  - Adding Non-linearities

Equation for Feed Forward Layer

$$\boxed{\begin{aligned} m_i &= MLP(\text{output}_i) \\ &= W_2 * \text{ReLU}(W_1 \times \text{output}_i + b_1) + b_2 \end{aligned}}$$

  - Residual Connection
  - Layer Normalization
  - Scaled Dot Product Attention

$$\textbf{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \textbf{softmax}\left(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}}\right)\boldsymbol{V}$$

$$\because \frac{\partial S_i}{\partial z_i} = S_i(1 - S_i)$$

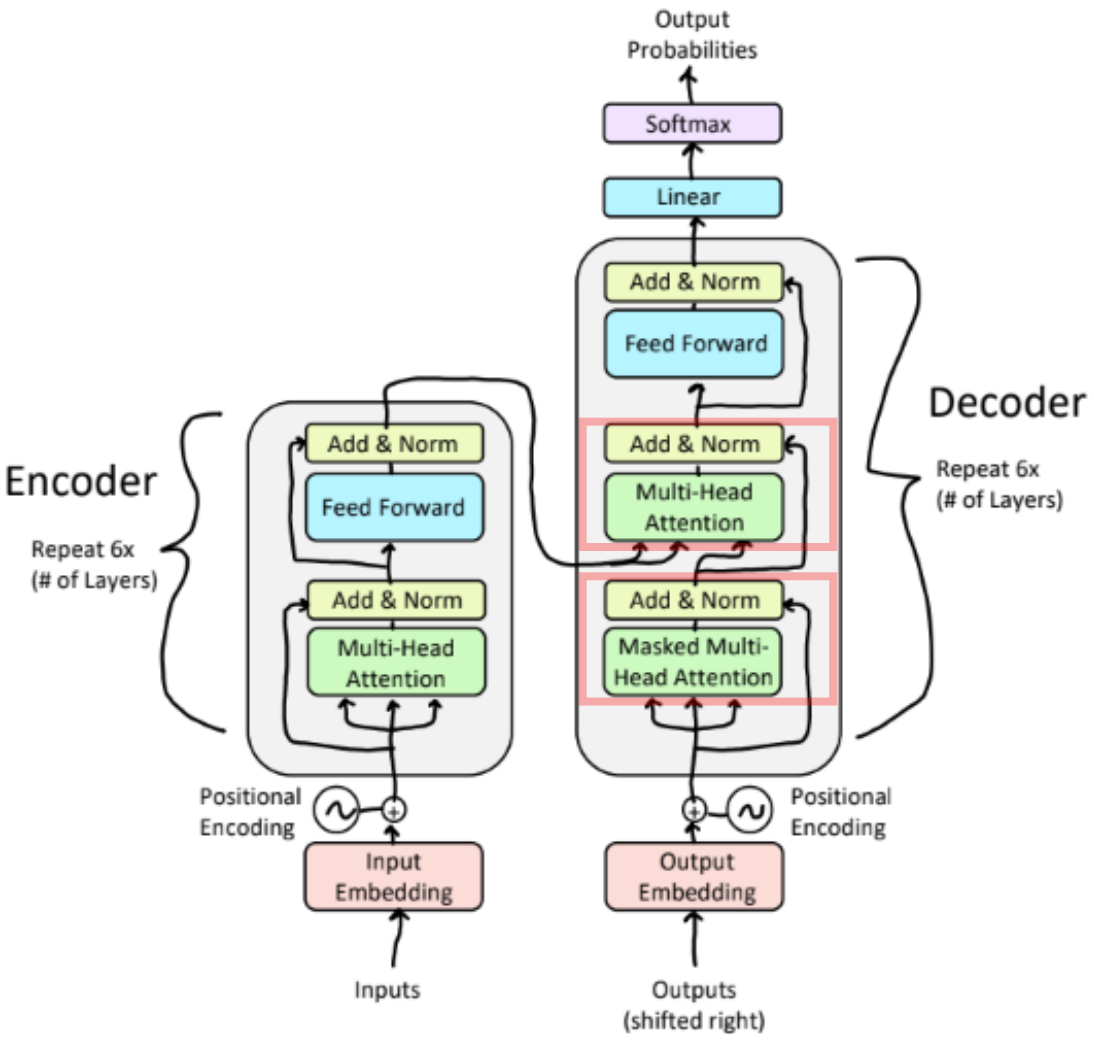
  - Multi-Head Attention

# Lecture 8

- **Self-Attention in Decoder**
  - Masking Future Information

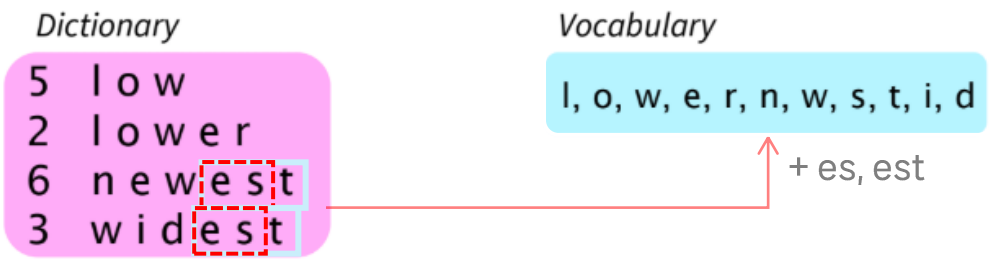
$$e_{ij} = \begin{cases} q_i^\top k_j, j < i \\ -\infty, j \geq i \end{cases} \rightarrow \text{Attention Score} = 0$$

  - Encoder-Decoder Attention (in transformer)
    - Query($\boldsymbol{Q}$):
      - From Decoder
      - Previous Results
    - Key($\boldsymbol{K}$):
      - From Encoder
      - Source Word's Index
    - Value($\boldsymbol{V}$):
      - From Encoder
      - Source Word's Information



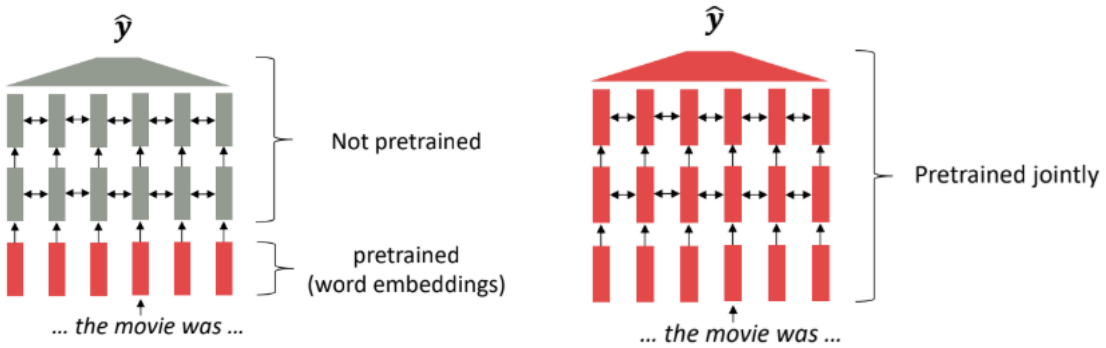
# Lecture 9: Pretraining

- **Subword Modeling**
  - Byte-pair Encoding




  - Split the Words

- **Pretraining**
  - Pretrain Word Embeddings
  - Pretrain Whole Model
    - Representations ↑
    - Parameter Initialization (Help Downstream Tasks)
    - Probability Distribution

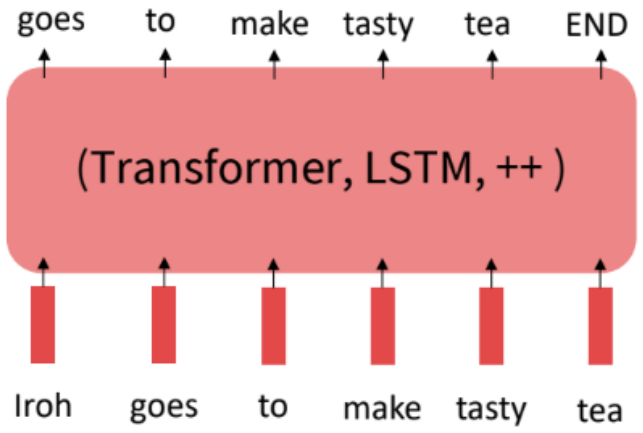


# Lecture 9

- **Pretraining/Finetuning**

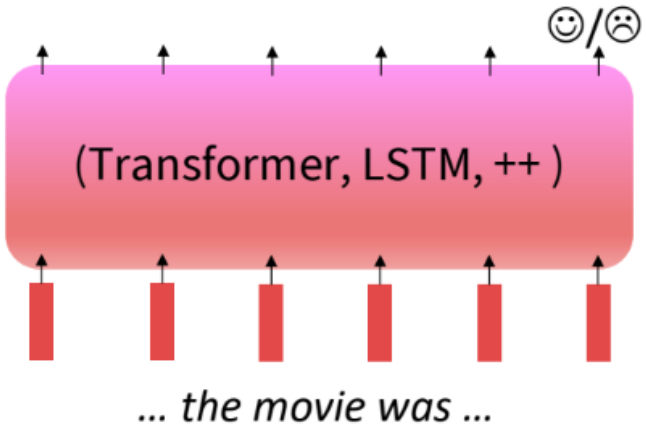
**Pretraining**

- Learn General things
- Supervised Learning (w/ Mask & Predict)



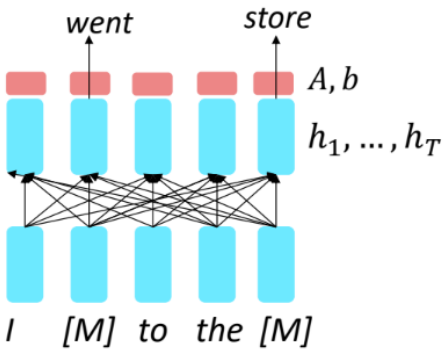

**Finetune**

- Adapt to Tasks
- Small Sized Data

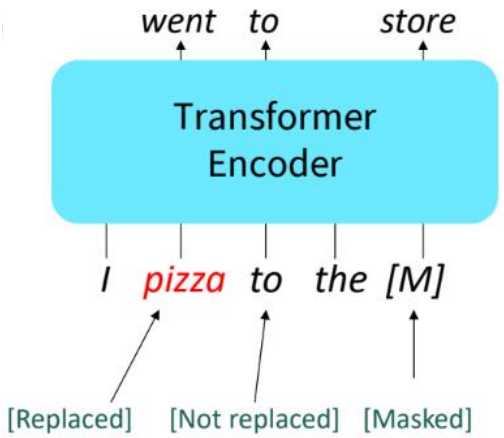



- **3 Types of Pretraining**
  - Encoders
    - Can see Forward Words
    - Do Masking and Predict

    ⇒ *Masked LM*




  - BERT
    - Select 15% of Subword Tokens
      - 80% -> [MASK]
      - 10% -> Randomly Change
      - 10% -> Unchanged
    - Next Sentence Prediction



# Lecture 9

- Encoder-Decoders

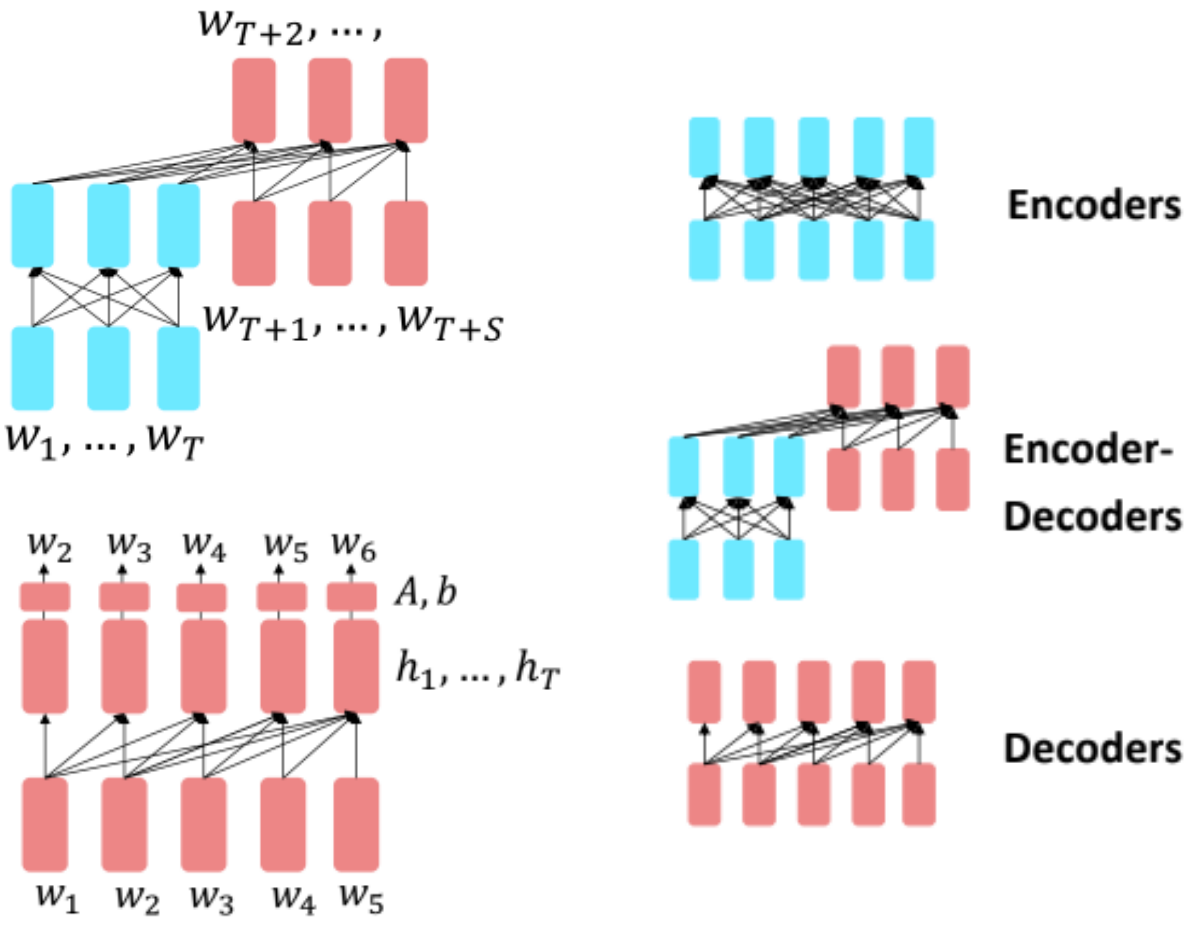
$$h_1, \dots, h_T = \text{Encoder}(w_1, \dots, w_T)$$
$$h_{T+1}, \dots, h_{T+S} = Decoder(w_{T+1}, \dots, w_{T+S}, h_1, \dots, h_T)$$
$$y_i \sim Ah_i + b, i > T$$

Original text
Thank you for inviting me to your party last week.

Inputs
Thank you <X> me to your party <Y> week.

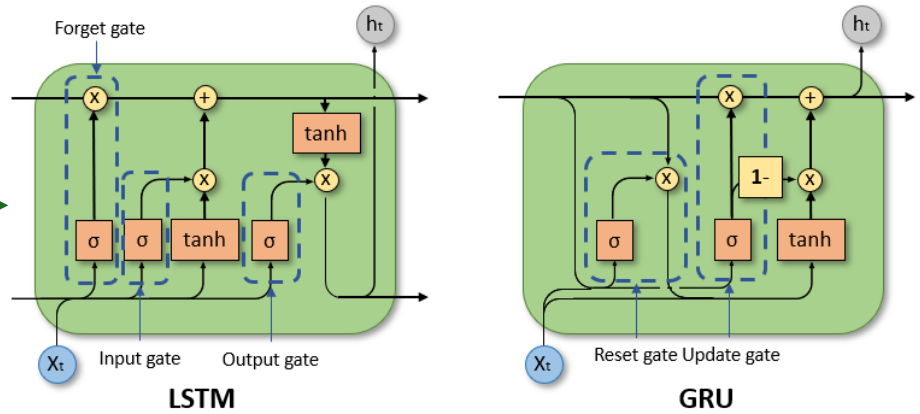
Targets
<X> for inviting <Y> last <Z>

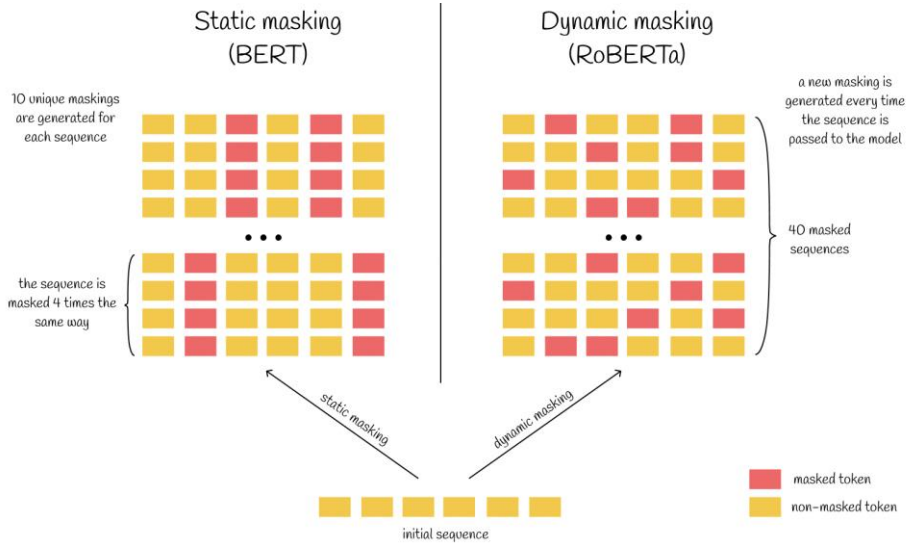
- Decoders
  - Good for Scaling
  - Zero-shot Learning
  - Emergent Ability

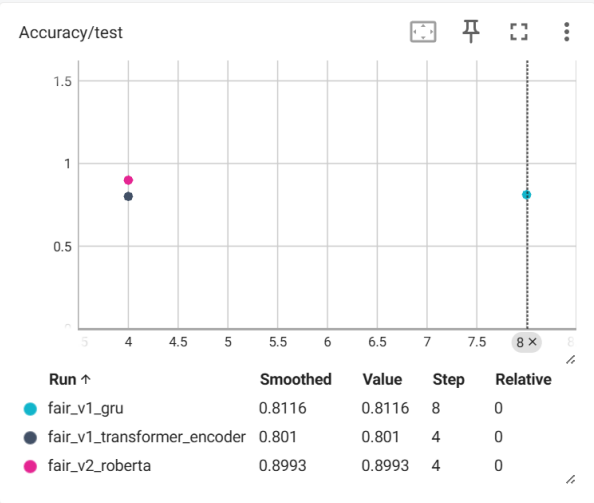
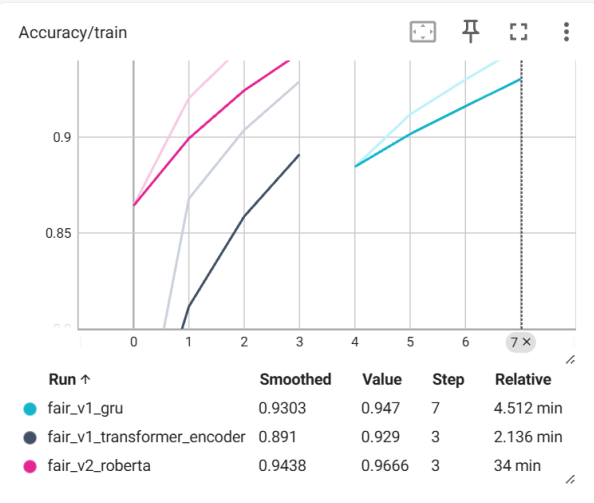
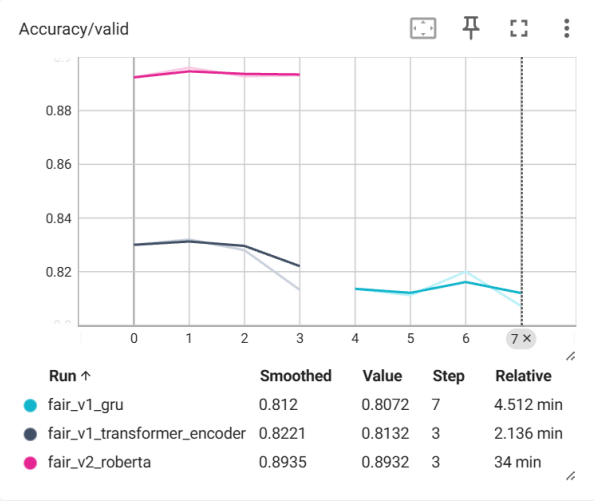


# Pytorch Practice

- **Dataset**
  - IMDB Review Dataset
- **Model**
  - LSTM/GRU
  - Transformer Encoder
  - BERT/RoBERTa
    - Link: Installation
- **Goal**
  - Over 80% Accuracy



# Pytorch Practice

- **Utility**
- **Hyperparameter**
  - Epochs: 4
  - Batch: 16
  - Max_len: 128
  - Grad_clip: 1.0
  - Weight_decay: 0.01
  - Lr: 1e-4/2e-5
- **Training Result**
  - GRU: Over 80% Acc (8 epochs)
  - Transformer: Over 80% Acc (4 epochs)
  - RoBERTa: Over 90% Acc (4 epochs)